

AI技术调研与场景分析

2023141480038

周佳楠

项目名称：Chat Box — 基于Spring Boot的AI对话平台

一、项目背景与选题依据

Chat Box是基于Spring Boot框架构建的AI对话平台，采用前后端分离架构，集成Spring Security、MyBatis-Plus、PostgreSQL、Redis、Kafka、Spring AI等技术，实现了用户注册登录、JWT认证、API Key管理、会话管理和消息收发等核心功能。在第一次课后作业中，本项目已参考ISO/IEC 12207[6]等国际标准，从软件过程过程、项目管理过程、支持过程和组织过程四个维度进行了系统化的过程定义，但在传统人工模式下，各阶段活动均存在效率和质量方面的提升空间。

2024-2026年间，以GitHub Copilot、Cursor、Claude等为代表的AI编程工具已广泛覆盖软件开发生命周期各阶段[3][7]。根据中国信通院发布的《智能化软件工程技术和应用要求》[1]和《AI4SE行业现状调查报告》[4]，AI技术在需求捕获与文档生成、架构推荐与建模、代码生成与安全审查、测试用例生成与缺陷预测、异常检测与日志分析等方面展现出显著的应用价值[1][4]。本研究在此基础上，以Chat Box项目为载体，系统分析AI在各阶段的应用场景，识别关键改进节点，并通过原型验证确认改进效果。

二、AI技术在SDLC各阶段的应用场景分析

2.1 需求分析阶段

传统需求分析依赖人工访谈和手动编写文档，耗时且易遗漏边界条件。在该阶段，LLM可发挥以下作用：

(1) 需求自动捕获与结构化表达：通过将用户访谈录音转写为文本后输入LLM，可自动识别用户注册、JWT认证等功能性需求以及响应时间、并发量等非功能性需求，并按标准模板生成结构化需求文档草稿[1]。据AI4SE行业调查报告，该方式可减少约60%的文档编写时间[4]，同时识别人工容易忽略的隐含需求，如JWT令牌轮换策略、API Key权限粒度等安全需求。

(2) 需求一致性验证与冲突检测：将需求说明书输入LLM，可自动检测需求条目间的矛盾，如"令牌永不过期"与"安全策略要求定期重新认证"之间的冲突，生成一致性验证报告。LLM还可参考同类开源项目，如Spring AI官方示例[8]，提供对比分析，提升需求规格的完整性和合理性。

(3) 需求优先级排序辅助：AI可根据业务影响度、技术复杂度、依赖关系等维度对需求进行自动分类和优先级排序，辅助产品负责人制定迭代计划[5]。

2.2 系统设计阶段

设计阶段涵盖架构设计、数据库设计、接口规范制定和安全设计等关键活动。

(1) 架构设计与建模：Copilot等IDE嵌入式AI工具可根据需求描述自动生成Mermaid或PlantUML格式的架构图初稿[3]，加速高层设计输出。AI还可根据实体关系推导ER图并生成对应的DDL脚本，减少手动建模工作。

(2) 接口规范自动生成：LLM可根据Controller方法签名和业务描述，自动生成符合OpenAPI 3.0规范的Swagger接口定义文档，确保前后端开发人员基于一致、准确的接口约定并行开发[7]。

(3) 安全设计审查：AI可对JWT认证流程、AES加密方案、Spring Security配置进行威胁建模，识别硬编码密钥、弱加密算法、权限绕过等设计缺陷。据Gtest 2025年报告，AI辅助安全审查可发现约40%人工审查遗漏的安全问题[2]。

2.3 代码实现阶段

代码实现阶段是当前AI应用最为广泛和成熟的环节。

(1) AI辅助代码生成：Chat Box项目采用Controller-Service-Mapper分层模式，代码模板化程度高，非常适合AI辅助生成。开发人员只需编写接口签名和关键注释，AI即可生成完整的业务逻辑实现和单元测试骨架[3]。GitHub 2024年数据显示，Copilot可帮助开发者以快46%的速度编写代码[3]。

(2) 智能代码审查：AI工具可自动检测SQL注入、XSS攻击、权限控制缺失等安全漏洞，检查代码规范合规性，评估代码复杂度，包括圈复杂度和认知复杂度，标记需要重构的代码区域[1][4]。每次Pull Request提交时，AI可自动分析变更影响范围并生成审查预读报告。

(3) 框架学习与最佳实践推荐：Spring AI作为较新的框架，官方文档示例有限。AI可根据官方文档自动生成符合项目上下文的最佳实践代码片段，降低学习成本[8]。

2.4 测试验证阶段

AI在测试阶段提供从用例生成到缺陷预测的全链路支持[2][4]。

(1) 智能测试用例生成：使用LLM测试工具可根据方法签名自动生成覆盖正常路径、边界条件和异常路径的测试用例集，结合参数化测试框架实现高覆盖率测试。据Gtest 2025年报告，AI辅助测

试可将代码覆盖率提升15-25个百分点，测试脚本编写时间减少约50%[2]。

(2) AI安全测试：AI可自动模拟SQL注入、XSS攻击、CSRF攻击等安全测试场景，生成针对性的攻击载荷和测试脚本，弥补人工安全测试覆盖面不足的问题[5]。

(3) 缺陷预测与风险识别：AI基于代码复杂度指标、历史缺陷数据和变更频率，可识别高风险模块并建议重点测试资源分配，实现基于风险的测试策略优化[1]。

2.5 运维监控阶段

传统运维依赖阈值告警和人工巡检，响应滞后且容易漏报[4][5]。

(1) 智能异常检测：AI运维引擎通过学习系统日志模式、应用指标趋势和用户行为基线，建立正常状态模型，在状态偏离时自动触发告警并附带上下文诊断信息，实现从被动响应向主动预防的转变[5]。

(2) 智能故障排查：在响应延迟升高时，AI可自动执行数据库慢查询分析、Redis缓存命中率检查、外部LLM API响应时间监测等诊断任务，快速定位瓶颈并给出调优建议[4]。

(3) 日志自动化分析：AI可对海量日志进行模式挖掘和异常聚类，提取高频错误模式和根因分析报告，辅助开发运维团队高效定位和修复问题。

2.6 AI应用场景匹配总结

综上所述，Chat Box项目在全生命周期中可识别十余个AI应用节点：

SDLC阶段	AI应用场景	主要AI能力	预期价值
需求分析	LLM辅助需求捕获与文档生成[1][4]	自然语言理解、结构化生成	减少文档时间约60%
系统设计	AI辅助架构图、ER图、DDL生成[3][8]	代码理解、模式识别	加速设计输出
系统设计	AI安全威胁建模与审查[2]	安全知识库、漏洞识别	发现约40%人工遗漏问题
代码实现	AI辅助代码生成[3][7]	代码补全、模板生成	编码速度提升约46%
代码实现	AI智能代码审查[1][4]	静态分析、漏洞扫描	提前发现安全与质量问题
测试验证	AI测试用例生成[2]	测试生成、边界分析	覆盖率提升15-25pp

测试验证	AI缺陷预测[1][4]	机器学习、统计分析	优化测试资源分配
运维监控	AI异常检测与故障排查[4][5]	异常检测、模式挖掘	减少故障定位时间

这些场景覆盖了SDLC五个核心阶段，为后续的过程改进方案设计和原型验证提供了明确的目标和方向。

参考文献

- [1] 中国信通院. 智能化软件工程技术和应用要求[R]. 2024.
- [2] Gtest全球软件测试技术峰会报告[R]. 2025.
- [3] GitHub. Copilot官方文档[EB/OL]. 2024-2026.
- [4] AI4SE行业现状调查报告[R]. 2024.
- [5] 生成式AI在软件工程中的应用与挑战[J]. 计算机学报, 2025.
- [6] ISO/IEC 12207:2017. Systems and software engineering[S]. 2017.
- [7] Anthropic. Claude API Documentation[EB/OL]. 2024-2026.
- [8] Spring AI Reference Documentation[EB/OL]. 2025.